Lab 05: An Alarming Amount of Pins

ECE 484: Embedded Systems Design

Grade	Score	Signature
Attendance: James Nippa		
Attendance: Gabe Littelken		
Attendance:		
Demonstration		

Overview

In this lab, you will design an embedded system that will act as an alarm system. Upon startup or reset, the system will countdown from 10 seconds. Once the count reaches 0 seconds, a red LED will be lit and a passive buzzer will sound. The frequency of the passive buzzer will controlled by a quadrature encoder. Finally, using a separate microcontroller, the status of the alarm will be displayed on an LCD module.

Deliverables

1. System Demonstration (10 points)
2. Completed Worksheet submitted to Canvas (5 points)
3. Commented Source Code submitted to Canvas (5 points)

Instructions

The system consists of two microcontrollers, two seven-segment displays, a rotary encoder, a push button, an LED, a passive buzzer, and an LCD module along with all corresponding interfacing circuitry. To complete this system, you will need to wire and program two sub-systems that are connected to each microcontroller.

Sub-system 1: Alarm Client

The "alarm client" subsystem consists of the seven-segment displays, encoder, push button, LED, and passive buzzer circuitry. In this sub-system, you will configure Timer 0, Timer 1, and Timer 2.

The two 8-bit timers, Timer 0 and Timer 2, will handle peripheral timing functions in this subsystem. Timer 0 will be configured to trigger an interrupt every 1ms and you will use time multiplexing to decrement the alarm count every 1 second. The alarm count variable will be used in the Timer 1 ISR to display the current alarm count in **decimal form**. The passive buzzer, along with the LED, will be activated when the alarm count reaches zero. Make sure that the alarm count remains zero, the LED remains lit, and the buzzer sounds, for as long as the push button is not pressed. Timer 2 will be used to generate square waves for the passive buzzer at a frequency defined by the value written to OCR2A. The value of OCR2A will be adjusted in the Timer 1 ISR.

The 16-bit timer, Timer 1, will be configured to sample the two channels of the encoder and the state of the push button at a tenth of the same sampling rate you used in the previous lab. The

Scanned with CamScanner

reduced Timer 1 sampling rate is needed to compensate for increased length of the Timer 1 ISR. Failure to reduce the Timer 1 sampling rate will cause the Timer 1 ISR to hoard the system clock such that other interrupts will be unable to trigger. The three inputs will be connected to the GPIO of your choice and will need to be passed through the `softwareDebounce` routine. For proper encoder operation, make sure that the encoder channel inputs are in the 2 LSB of the variable you pass to the `softwareDebounce` function. After debouncing the inputs, you will need to increment/decrement an encoder count and check for a falling edge on the push button. A detected falling edge on the push button should reset the alarm count back to 10 seconds and disable the LED and buzzer. Meanwhile, the encoder count will be used to adjust the frequency of the passive buzzer. **It is important to note that the passive buzzer will be disabled by disabling the Timer 2 output compare match interrupt.** To adjust the frequency of the encoder, you will use the `encodercount` variable. For every fourth increment/decrement of the encoder (corresponding to the tactile "click"), the frequency output by the encoder should increment/decrement by 250Hz. The minimum frequency of the passive buzzer should be 1.5kHz while the maximum frequency of the passive buzzer should be 2.5kHz. Your code should loop through the five tones (HINT: use integer division to check when to change frequencies). This will include handling overflows of the `encoderCount` variable (`encoderCount` < 0 will overflow to 19 and `encoderCount` > 19 will overflow to 0). After this, you will use time multiplexing to toggle the seven segment displays and output the current alarm count. Finally, your "alarm client" sub-system should send a high signal to the "alarm HQ" sub-system when the alarm count has reached zero to communicate that the alarm has been tripped.

Sub-system 2: Alarm HQ

The second sub-system consists of the LCD module circuitry that will output the current status of the alarm and how many times the alarm has tripped. If the alarm is off, the first line of the LCD should read "ALARM: OFF". Conversely, when the alarm is tripped, the first line of the LCD should read "ALARM: ON". To determine the alarm state, you will configure a pin change interrupt on a GPIO of your choice. In the pin change ISR, you will need to determine the state of the signal and output the proper message to the LCD. You will also need to record the number of times the alarm has tripped and display that count on the second line of the LCD. The second line of the LCD should read "TRIPS: XYZ" where XYZ is the number of times the alarm has tripped, expressed as a three digit decimal number.

For this sub-system, you will connect RS, RW, and E on the LCD module to PINB0, PINB1, and PINB2, respectively. The data lines on the LCD module, D0-D7, will be connected to PIND0-D7, respectively. Interfacing circuitry for the contrast and LED back light will also need to be constructed. A circuit diagram for the LCD is illustrated in Figure 1.

You will need to write separate programs for each sub-system. For both sub-system programs, you are not permitted to write any code in the `while(1)` loop.

Scanned with CamScanner

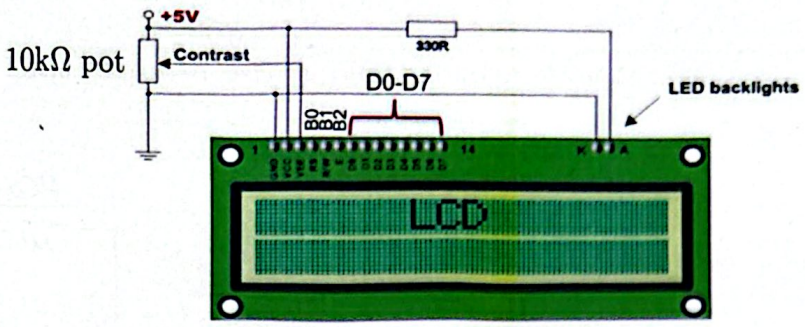


Figure 1: LCD Module Schematic

Demonstration Rubric

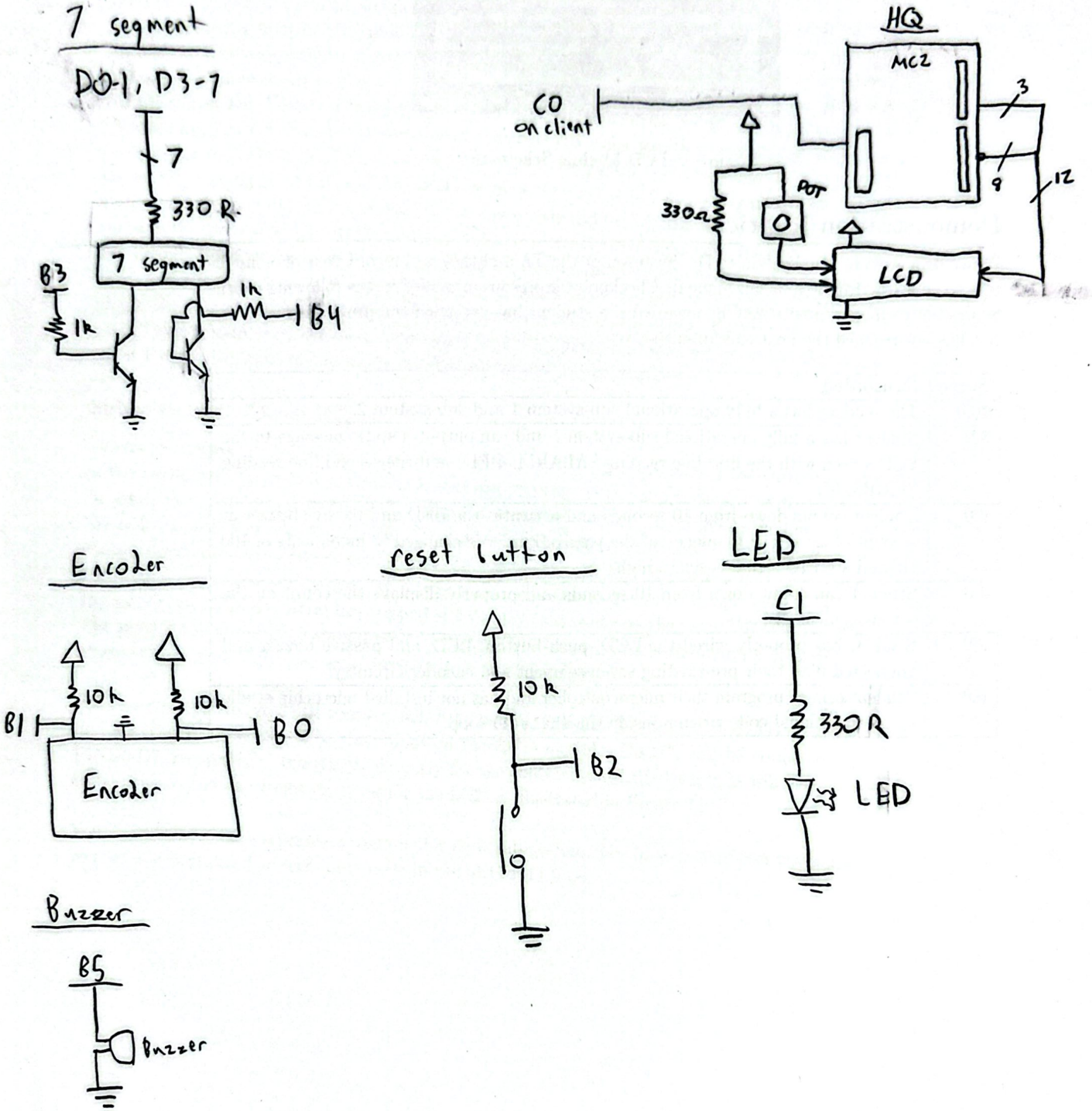
Please demonstrate your system to Dr. Swanson or the TA and they will record your demonstration score at the top of your lab manual. Checkpoint scores are provided in the following rubric. Scores between checkpoints may be awarded if a student has exceeded the previous checkpoint but has not reached the next checkpoint(s).

Score	Reasoning
10.0	The student has a fully operational sub-system 1 and sub-system 2.
8.0	Student has a fully operational sub-system 1 and can output a static message to the LCD screen with the first line reading "ALARM: OFF" and the second line reading "TRIPS: 0".
6.0	Student counts down from 10 seconds and activates the LED and passive buzzer at a count of zero. The frequency of the passive buzzer is changed in increments of 100 Hz and a GPIO signal is set to high.
4.0	Student can count down from 10 seconds and properly displays the count on the seven-segment displays.
2.0	Student has properly wired the LCD, push-button, LED, and passive buzzer and connected it to their pre-existing seven-segment and encoder circuitry.
0.0	Student cannot program their microcontroller and has not installed microchip studio or has functional code written inside the `while(1)` loop

Scanned with CamScanner

Circuit Diagram

Draw a circuit diagram for your system. Make sure to label all major modules, resistance values, and signal lines.



Scanned with CamScanner

Design Questions

1. Using the datasheet for the HJ1602A LCD module, write the function set command (PORTD and PORTB configurations) to enable 4-bit, single line mode, with 5x10 digits. Assume that you are using the same wiring used in this lab.

$$PORTD = \underbrace{(0 << PIND4)}_{4\ bits} \mid \underbrace{(1 << PIND2)}_{5x10};$$

2. What is one benefit of using 4-bit mode and one drawback of using 4-bit mode (as opposed to 8-bit mode).

Pro: More GPIO available for other things

Con: More time required to set a char at an address as the code is an 8-bit sequence. Therefore, it would roughly 2x slower.

3. Using the datasheet for the HJ1602A LCD module, what is the difference between the clear display function and the return home function?

- Clear display will clear all char at every address and then return the cursor to the home address.
- Return home just returns the cursor to the home address.

Scanned with CamScanner

4. When the buzzer is not activated, you can still hear a faint tone coming out of the buzzer. What causes this faint tone to sound?

Either crosstalk from other wires or a system heartbeat. Since a buzzer has a resistance of 8 Ω, even small voltages can cause a small amount of current to flow.

Scanned with CamScanner